

Case Study on the Analysis of SDLC Models Applied to a Real-World Software Application

Aim

To analyze different Software Development Life Cycle (SDLC) models and identify the most suitable model for developing a Smart Parking Management System.

Objective

- To understand the different SDLC models.
- To study the software development process of a Smart Parking Management System.
- To compare the advantages and disadvantages of various SDLC models.
- To recommend the most appropriate SDLC model for the application.

Selected Software Application

Smart Parking Management System

Overview

A Smart Parking Management System is a software application that helps drivers locate, reserve, and pay for parking spaces efficiently. It also enables parking administrators to monitor parking occupancy, manage users, and generate reports. The system reduces traffic congestion, saves time, and improves parking utilization.

Functional Requirements

User Module

- User Registration
- Secure Login
- Search Available Parking Slots
- Reserve Parking Space
- Online Payment
- View Booking History
- Receive Parking Notifications

Parking Operator Module

- Manage Parking Slots
- Update Slot Availability
- Monitor Vehicle Entry and Exit
- View Daily Parking Records

Admin Module

- Manage Users
- Manage Parking Locations
- Generate Reports
- Monitor Revenue
- System Configuration

SDLC Models Analysis

SDLC Model	Suitability	Advantages	Limitations
Waterfall	Low	Easy to understand and manage	Difficult to modify requirements
Prototype	Medium	Better understanding of user needs	Additional development time
Spiral	High	Effective risk management	Expensive and complex
Agile	Very High	Flexible, fast development, continuous feedback	Requires active collaboration
DevOps	High	Continuous integration and deployment	Requires automation infrastructure

Why Agile is the Best Choice

The Agile model is the most suitable for developing a Smart Parking Management System because:

- Parking requirements may change based on user feedback.
- New payment methods can be added easily.
- Smart sensors and IoT devices require frequent software updates.
- Security improvements and bug fixes can be released quickly.
- Continuous testing improves system reliability.

Agile Development Process

Requirement Analysis



Sprint Planning



System Design



Development



Testing



Deployment



User Feedback



Next Sprint

SDLC Phases

1. Requirement Analysis

Activities

- Study user requirements
- Analyze parking slot management
- Define payment system requirements
- Identify notification requirements

Deliverable

- Software Requirement Specification (SRS)

2. System Design

Design Includes

- Database Design
- User Interface Design
- System Architecture
- API Design

Deliverables

- UML Diagrams
- Database Schema
- Wireframes

3. Development

Modules Developed

- User Authentication Module
- Parking Slot Management Module
- Booking Module
- Payment Module
- Notification Module

- Admin Dashboard

Technologies Used

Frontend

- HTML
- CSS
- JavaScript

Backend

- Python (Flask) / Node.js

Database

- MySQL

4. Testing

Types of Testing

- Unit Testing
- Integration Testing
- System Testing
- User Acceptance Testing (UAT)

Sample Test Cases

Test Case	Expected Result
User Login	Successful login with valid credentials
Search Parking Slot	Available slots displayed correctly
Reserve Parking	Parking slot reserved successfully
Online Payment	Payment completed successfully
Vehicle Exit	Parking slot becomes available again

5. Deployment

Deployment Methods

- Cloud Deployment
- Mobile Application Deployment
- Web Server Deployment

Cloud Platforms

- Amazon Web Services (AWS)
- Microsoft Azure
- Google Cloud Platform (GCP)

6. Maintenance

Activities

- Bug Fixes
- Security Updates
- Performance Optimization
- Database Backup
- Feature Enhancements

Comparison of SDLC Models

Criteria	Waterfall	Prototype	Spiral	Agile	DevOps
Requirement Changes	Poor	Good	Excellent	Excellent	Excellent
Customer Feedback	Low	High	High	Very High	Very High
Risk Management	Low	Medium	High	Medium	High

Development Speed	Slow	Medium	Medium	Fast	Very Fast
Cost	Low	Medium	High	Medium	Medium
Suitable for Smart Parking System	No	Partially	Yes	Best	Best

Results

- The Smart Parking Management System requires frequent software updates and continuous improvements.
- Agile enables rapid development through iterative sprints and regular customer feedback.
- DevOps supports automated testing, deployment, and monitoring.
- The combination of Agile and DevOps ensures reliable, scalable, and high-quality software.

Conclusion

This case study evaluated different SDLC models for a Smart Parking Management System. Among the models analyzed, the **Agile SDLC model** is the most suitable because it supports flexibility, faster development, and continuous user feedback. Integrating Agile with DevOps practices further improves deployment efficiency, software quality, and system maintenance, making it an ideal approach for smart parking applications.